AtlasIngest - System Architecture

1. Scale Strategy for 500,000+ Records

To handle 500,000+ records, AtlasIngest uses asynchronous I/O across the entire pipeline. Data collection utilizes aiohttp for high-concurrency fetching with connection pooling. Persistence leverages asyncpg and SQLAlchemy 2.0 to stream inserts efficiently to PostgreSQL. Crawling is partitioned into distinct phases (Discovery, Extraction, Structuring) to prevent bottlenecks.

2. Async Crawling Architecture

Implemented via a custom CrawlerEngine leveraging Python's asyncio and aiohttp. Concurrency is tightly controlled using semaphores (global limit of 20 concurrent requests, per-host limits of 5). This avoids resource exhaustion and ensures respectful crawling.

3. 413 Handling

Implemented: The async HTTP client intercepts HTTP 413 Payload Too Large responses. Currently, if a payload exceeds configured limits, the crawler logs the failure. For massive APIs, adapters paginate data to ensure response chunks remain small.

4. 429 Handling

Implemented: A deterministic RetryEngine gracefully handles HTTP 429 Too Many Requests. It respects the 'Retry-After' header and automatically sleeps for the requested duration before retrying. If the header is missing, it falls back to truncated exponential backoff (e.g., up to 30 seconds).

5. Freshness / Deduplication

Implemented: A SHA-256 hash of the raw response payload is calculated on ingestion. If the hash matches an existing crawl run for that URL, redundant processing is skipped. The crawler natively enforces unique entity constraints to ensure exactly-once insertion for deduplicated data.

6. PostgreSQL Storage

Implemented: The system uses PostgreSQL for scalable relational storage. It strictly separates raw data ('raw_documents') from structured data ('research_papers', 'startups', 'products') to ensure no data is lost. All schema definitions use declarative SQLAlchemy Base with strongly-typed columns and UUID primary keys.

7. Entity Resolution

Implemented: The entity_normalization.py pipeline standardizes entities (like companies and startups) deterministically. It lowercases names, strips punctuation, and removes common corporate suffixes (Inc, LLC, Corp) before inserting into the database. Database UNIQUE constraints enforce exactly-one entry per canonical entity name.

8. LLM Fallback Architecture for Phase 5 (PLANNED)

PLANNED (Not Yet Implemented): Phase 5 will introduce a secondary unstructured pipeline. If deterministic extraction fails to yield structured schemas, raw_documents will be routed to an LLM extraction queue. The LLM (e.g., via prompt-engineered structured outputs) will fill the missing fields, allowing robust extraction for heterogeneous data

sources.

9. Anti-Bot Strategy

Implemented: The system employs standard anti-blocking mechanisms including customized User-Agent headers, connection keep-alives, strict per-domain concurrency limits, and jittered exponential backoff retries. It strictly falls back to server-rendered HTML payloads (e.g., Next.js data-page) when REST APIs respond with 403 Forbidden.

10. Provenance / Data Quality

Implemented: Every structured record maintains strict provenance mapping back to its origin URL. The assignment explicitly enforces deterministic rule-based extractions. Nulls are preferred over hallucinated data. If a relationship is ambiguous (e.g., product owner unresolved), the system explicitly labels it as UNRESOLVED and rejects the insertion. Pydantic strictly validates all models.